JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Class AsynchronousSocketChannel

java.lang.Object
  java.nio.channels.AsynchronousSocketChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `AsynchronousByteChannel`, `AsynchronousChannel`, `Channel`, `NetworkChannel`

```
public abstract class AsynchronousSocketChannel
extends Object
implements AsynchronousByteChannel, NetworkChannel
```

An asynchronous channel for stream-oriented connecting sockets.

Asynchronous socket channels are created in one of two ways. A newly-created `AsynchronousSocketChannel` is created by invoking one of the `open` methods defined by this class. A newly-created channel is open but not yet connected. A connected `AsynchronousSocketChannel` is created when a connection is made to the socket of an `AsynchronousServerSocketChannel`. It is not possible to create an asynchronous socket channel for an arbitrary, pre-existing `socket`.

A newly-created channel is connected by invoking its `connect` method; once connected, a channel remains connected until it is closed. Whether or not a socket channel is connected may be determined by invoking its `getRemoteAddress` method. An attempt to invoke an I/O operation upon an unconnected channel will cause a `NotYetConnectedException` to be thrown.

Channels of this type are safe for use by multiple concurrent threads. They support concurrent reading and writing, though at most one read operation and one write operation can be outstanding at any time. If a thread initiates a read operation before a previous read operation has completed then a `ReadPendingException` will be thrown. Similarly, an attempt to initiate a write operation before a previous write has completed will throw a `WritePendingException`.

Socket options are configured using the `setOption` method. Asynchronous socket channels support the following options:

Option Name	Description
`SO_SNDBUF`	The size of the socket send buffer
`SO_RCVBUF`	The size of the socket receive buffer
`SO_KEEPALIVE`	Keep connection alive
`SO_REUSEADDR`	Re-use address
`TCP_NODELAY`	Disable the Nagle algorithm

Additional (implementation specific) options may also be supported.

Timeouts

The `read` and `write` methods defined by this class allow a timeout to be specified when initiating a read or write operation. If the timeout elapses before an operation completes then the operation completes with the exception `InterruptedByTimeoutException`. A timeout may leave the channel, or the underlying connection, in an inconsistent state. Where the implementation cannot guarantee that bytes have not been read from the channel then it puts the channel into an implementation specific *error state*. A subsequent attempt to initiate a `read` operation causes an unspecified runtime exception to be thrown. Similarly if a `write` operation times out and the implementation cannot guarantee bytes have not been written to the channel then further attempts to `write` to the channel cause an unspecified runtime exception to be thrown. When a timeout elapses then the state of the `ByteBuffer`, or the sequence of buffers, for the I/O operation is not defined. Buffers should be discarded or at least care must be taken to ensure that the buffers are not accessed while the channel remains open. All methods that accept timeout parameters treat values less than or equal to zero to mean that the I/O operation does not timeout.

Since:

`1.7`

Constructor Summary

Constructors

Modifier	Constructor	Descr
`protected`	`AsynchronousSocketChannel(AsynchronousChannelProvider provider)`	Initia a new instan of thi class

Method Summary

Concrete Methods | **Abstract**

Modifier and Type	Method	Desc
abstract **AsynchronousSocketChannel**	**bind**(**SocketAddress** local)	Bind chan sock loca addr
abstract **Future**<**Void**>	**connect**(**SocketAddress** remote)	Con chan
abstract <A> void	**connect**(**SocketAddress** remote, A attachment, **CompletionHandler**<**Void**,? super A> handler)	Con chan
abstract **SocketAddress**	**getLocalAddress**()	Retu sock addr this sock bou
abstract **SocketAddress**	**getRemoteAddress**()	Retu rem addr whi chan sock con
static **AsynchronousSocketChannel**	**open**()	Ope asyn sock chan
static **AsynchronousSocketChannel**	**open**(**AsynchronousChannelGroup** group)	Ope asyn sock chan
AsynchronousChannelProvider	**provider**()	Retu prov crea chan

abstract **Future**<**Integer**>	**read**(**ByteBuffer** dst)	Rea sequ byte this into buff
abstract <A> void	**read**(**ByteBuffer**[] dsts, int offset, int length, long timeout, **TimeUnit** unit, A attachment, **CompletionHandler**<**Long**,? super A> handler)	Rea sequ byte this into subs of th buff
<A> void	**read**(**ByteBuffer** dst, A attachment, **CompletionHandler**<**Integer**,? super A> handler)	Rea sequ byte this into buff
abstract <A> void	**read**(**ByteBuffer** dst, long timeout, **TimeUnit** unit, A attachment, **CompletionHandler**<**Integer**,? super A> handler)	Rea sequ byte this into buff
abstract <T> **AsynchronousSocketChannel**	**setOption**(**SocketOption**<T> name, T value)	Sets valu sock opti
abstract **AsynchronousSocketChannel**	**shutdownInput**()	Shu the con for with clos cha
abstract **AsynchronousSocketChannel**	**shutdownOutput**()	Shu the con for with

		clos cha
abstract **Future**<**Integer**>	**write**(**ByteBuffer** src)	Writ sequ byte cha the buff
abstract <A> void	**write**(**ByteBuffer**[] srcs, int offset, int length, long timeout, **TimeUnit** unit, A attachment, **CompletionHandler**<**Long**,? super A> handler)	Writ sequ byte cha a subs of th buff
<A> void	**write**(**ByteBuffer** src, A attachment, **CompletionHandler**<**Integer**,? super A> handler)	Writ sequ byte cha the buff
abstract <A> void	**write**(**ByteBuffer** src, long timeout, **TimeUnit** unit, A attachment, **CompletionHandler**<**Integer**,? super A> handler)	Writ sequ byte cha the buff

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods inherited from interface java.nio.channels.AsynchronousChannel

close

Methods inherited from interface java.nio.channels.Channel

isOpen

Methods inherited from interface java.nio.channels.NetworkChannel

getOption, supportedOptions

Constructor Detail

AsynchronousSocketChannel

```
protected AsynchronousSocketChannel(AsynchronousChannelProvider provider)
```

Initializes a new instance of this class.

Parameters:

provider - The provider that created this channel

Method Detail

provider

```
public final AsynchronousChannelProvider provider()
```

Returns the provider that created this channel.

Returns:

The provider that created this channel

open

```
public static AsynchronousSocketChannel open(AsynchronousChannelGroup group)
                                      throws IOException
```

Opens an asynchronous socket channel.

The new channel is created by invoking the `openAsynchronousSocketChannel` method on the `AsynchronousChannelProvider` that created the group. If the group parameter is `null` then the resulting channel is created by the system-wide default provider, and bound to the *default group*.

Parameters:

group - The group to which the newly constructed channel should be bound, or null for the default group

Returns:

A new asynchronous socket channel

Throws:

ShutdownChannelGroupException - If the channel group is shutdown

IOException - If an I/O error occurs

open

```
public static AsynchronousSocketChannel open()
                                      throws IOException
```

Opens an asynchronous socket channel.

This method returns an asynchronous socket channel that is bound to the *default group*.This method is equivalent to evaluating the expression:

```
open((AsynchronousChannelGroup)null);
```

Returns:

A new asynchronous socket channel

Throws:

IOException - If an I/O error occurs

bind

```
public abstract AsynchronousSocketChannel bind(SocketAddress local)
                                        throws IOException
```

Description copied from interface: NetworkChannel

Binds the channel's socket to a local address.

This method is used to establish an association between the socket and a local address. Once an association is established then the socket remains bound until the channel is closed. If the `local` parameter has the value `null` then the socket will be bound to an address that is assigned automatically.

Specified by:

bind in interface NetworkChannel

Parameters:

local - The address to bind the socket, or null to bind the socket to an automatically assigned socket address

Returns:

This channel

Throws:

ConnectionPendingException - If a connection operation is already in progress on this channel

AlreadyBoundException - If the socket is already bound

UnsupportedAddressTypeException - If the type of the given address is not supported

ClosedChannelException - If the channel is closed

IOException - If some other I/O error occurs

SecurityException - If a security manager has been installed and its checkListen method denies the operation

See Also:

NetworkChannel.getLocalAddress()

setOption

```
public abstract <T> AsynchronousSocketChannel setOption(SocketOption<T> name,
                                                        T value)
                                                 throws IOException
```

Description copied from interface: NetworkChannel

Sets the value of a socket option.

Specified by:

setOption in interface NetworkChannel

Type Parameters:

T - The type of the socket option value

Parameters:

name - The socket option

value - The value of the socket option. A value of null may be a valid value for some socket options.

Returns:

This channel

Throws:

IllegalArgumentException - If the value is not a valid value for this socket option

ClosedChannelException - If this channel is closed

IOException - If an I/O error occurs

See Also:

StandardSocketOptions

shutdownInput

```
public abstract AsynchronousSocketChannel shutdownInput()
                                                  throws IOException
```

Shutdown the connection for reading without closing the channel.

Once shutdown for reading then further reads on the channel will return `-1`, the end-of-stream indication. If the input side of the connection is already shutdown then invoking this method has no effect. The effect on an outstanding read operation is system dependent and therefore not specified. The effect, if any, when there is data in the socket receive buffer that has not been read, or data arrives subsequently, is also system dependent.

Returns:

The channel

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

shutdownOutput

```
public abstract AsynchronousSocketChannel shutdownOutput()
                                                  throws IOException
```

Shutdown the connection for writing without closing the channel.

Once shutdown for writing then further attempts to write to the channel will throw `ClosedChannelException`. If the output side of the connection is already shutdown then invoking this method has no effect. The effect on an outstanding write operation is system dependent and therefore not specified.

Returns:

The channel

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

getRemoteAddress

```
public abstract SocketAddress getRemoteAddress()
                                          throws IOException
```

Returns the remote address to which this channel's socket is connected.

Where the channel is bound and connected to an Internet Protocol socket address then the return value from this method is of type `InetSocketAddress`.

Returns:

The remote address; null if the channel's socket is not connected

Throws:

`ClosedChannelException` - If the channel is closed

`IOException` - If an I/O error occurs

connect

```
public abstract <A> void connect(SocketAddress remote,
                                 A attachment,
                                 CompletionHandler<Void,? super A> handler)
```

Connects this channel.

This method initiates an operation to connect this channel. The `handler` parameter is a completion handler that is invoked when the connection is successfully established or connection cannot be established. If the connection cannot be established then the channel is closed.

This method performs exactly the same security checks as the `Socket` class. That is, if a security manager has been installed then this method verifies that its `checkConnect` method permits connecting to the address and port number of the given remote endpoint.

Type Parameters:

`A` - The type of the attachment

Parameters:

`remote` - The remote address to which this channel is to be connected

`attachment` - The object to attach to the I/O operation; can be null

`handler` - The handler for consuming the result

Throws:

UnresolvedAddressException - If the given remote address is not fully resolved

UnsupportedAddressTypeException - If the type of the given remote address is not supported

AlreadyConnectedException - If this channel is already connected

ConnectionPendingException - If a connection operation is already in progress on this channel

ShutdownChannelGroupException - If the channel group has terminated

SecurityException - If a security manager has been installed and it does not permit access to the given remote endpoint

See Also:

getRemoteAddress()

connect

```
public abstract Future<Void> connect(SocketAddress remote)
```

Connects this channel.

This method initiates an operation to connect this channel. This method behaves in exactly the same manner as the connect(SocketAddress, Object, CompletionHandler) method except that instead of specifying a completion handler, this method returns a Future representing the pending result. The Future's get method returns null on successful completion.

Parameters:

remote - The remote address to which this channel is to be connected

Returns:

A Future object representing the pending result

Throws:

UnresolvedAddressException - If the given remote address is not fully resolved

UnsupportedAddressTypeException - If the type of the given remote address is not supported

AlreadyConnectedException - If this channel is already connected

ConnectionPendingException - If a connection operation is already in progress on this channel

SecurityException - If a security manager has been installed and it does not permit access to the given remote endpoint

read

```
public abstract <A> void read(ByteBuffer dst,
                              long timeout,
                              TimeUnit unit,
                              A attachment,
                              CompletionHandler<Integer,? super A> handler)
```

Reads a sequence of bytes from this channel into the given buffer.

This method initiates an asynchronous read operation to read a sequence of bytes from this channel into the given buffer. The `handler` parameter is a completion handler that is invoked when the read operation completes (or fails). The result passed to the completion handler is the number of bytes read or `-1` if no bytes could be read because the channel has reached end-of-stream.

If a timeout is specified and the timeout elapses before the operation completes then the operation completes with the exception `InterruptedByTimeoutException`. Where a timeout occurs, and the implementation cannot guarantee that bytes have not been read, or will not be read from the channel into the given buffer, then further attempts to read from the channel will cause an unspecific runtime exception to be thrown.

Otherwise this method works in the same manner as the `AsynchronousByteChannel.read(ByteBuffer,Object,CompletionHandler)` method.

Type Parameters:

`A - The type of the attachment`

Parameters:

`dst - The buffer into which bytes are to be transferred`

`timeout - The maximum time for the I/O operation to complete`

`unit - The time unit of the timeout argument`

`attachment - The object to attach to the I/O operation; can be null`

`handler - The handler for consuming the result`

Throws:

`IllegalArgumentException - If the buffer is read-only`

`ReadPendingException - If a read operation is already in progress on this channel`

`NotYetConnectedException - If this channel is not yet connected`

`ShutdownChannelGroupException - If the channel group has terminated`

read

```
public final <A> void read(ByteBuffer dst,
                           A attachment,
                           CompletionHandler<Integer,? super A> handler)
```

Description copied from interface: AsynchronousByteChannel

Reads a sequence of bytes from this channel into the given buffer.

This method initiates an asynchronous read operation to read a sequence of bytes from this channel into the given buffer. The `handler` parameter is a completion handler that is invoked when the read operation completes (or fails). The result passed to the completion handler is the number of bytes read or `-1` if no bytes could be read because the channel has reached end-of-stream.

The read operation may read up to *r* bytes from the channel, where *r* is the number of bytes remaining in the buffer, that is, `dst.remaining()` at the time that the read is attempted. Where *r* is 0, the read operation completes immediately with a result of `0` without initiating an I/O operation.

Suppose that a byte sequence of length *n* is read, where `0` < *n* <= *r*. This byte sequence will be transferred into the buffer so that the first byte in the sequence is at index *p* and the last byte is at index *p* + *n* - 1, where *p* is the buffer's position at the moment the read is performed. Upon completion the buffer's position will be equal to *p* + *n*; its limit will not have changed.

Buffers are not safe for use by multiple concurrent threads so care should be taken to not access the buffer until the operation has completed.

This method may be invoked at any time. Some channel types may not allow more than one read to be outstanding at any given time. If a thread initiates a read operation before a previous read operation has completed then a `ReadPendingException` will be thrown.

Specified by:

`read` in interface `AsynchronousByteChannel`

Type Parameters:

`A` - The type of the attachment

Parameters:

`dst` - The buffer into which bytes are to be transferred

`attachment` - The object to attach to the I/O operation; can be `null`

`handler` - The completion handler

Throws:

`IllegalArgumentException` - If the buffer is read-only

ReadPendingException - If the channel does not allow more than one read to be outstanding and a previous read has not completed

NotYetConnectedException - If this channel is not yet connected

ShutdownChannelGroupException - If the channel group has terminated

read

```
public abstract Future<Integer> read(ByteBuffer dst)
```

Description copied from interface: AsynchronousByteChannel

Reads a sequence of bytes from this channel into the given buffer.

This method initiates an asynchronous read operation to read a sequence of bytes from this channel into the given buffer. The method behaves in exactly the same manner as the read(ByteBuffer,Object,CompletionHandler) method except that instead of specifying a completion handler, this method returns a Future representing the pending result. The Future's get method returns the number of bytes read or -1 if no bytes could be read because the channel has reached end-of-stream.

Specified by:

read in interface AsynchronousByteChannel

Parameters:

dst - The buffer into which bytes are to be transferred

Returns:

A Future representing the result of the operation

Throws:

IllegalArgumentException - If the buffer is read-only

ReadPendingException - If the channel does not allow more than one read to be outstanding and a previous read has not completed

NotYetConnectedException - If this channel is not yet connected

read

```
public abstract <A> void read(ByteBuffer[] dsts,
                              int offset,
                              int length,
                              long timeout,
                              TimeUnit unit,
                              A attachment,
                              CompletionHandler<Long,? super A> handler)
```

Reads a sequence of bytes from this channel into a subsequence of the given buffers. This operation, sometimes called a *scattering read*, is often useful when implementing network protocols that group data into segments consisting of one or more fixed-length headers followed by a variable-length body. The `handler` parameter is a completion handler that is invoked when the read operation completes (or fails). The result passed to the completion handler is the number of bytes read or `-1` if no bytes could be read because the channel has reached end-of-stream.

This method initiates a read of up to *r* bytes from this channel, where *r* is the total number of bytes remaining in the specified subsequence of the given buffer array, that is,

```
dsts[offset].remaining()
    + dsts[offset+1].remaining()
    + ... + dsts[offset+length-1].remaining()
```

at the moment that the read is attempted.

Suppose that a byte sequence of length *n* is read, where `0` < *n* <= *r*. Up to the first `dsts[offset].remaining()` bytes of this sequence are transferred into buffer `dsts[offset]`, up to the next `dsts[offset+1].remaining()` bytes are transferred into buffer `dsts[offset+1]`, and so forth, until the entire byte sequence is transferred into the given buffers. As many bytes as possible are transferred into each buffer, hence the final position of each updated buffer, except the last updated buffer, is guaranteed to be equal to that buffer's limit. The underlying operating system may impose a limit on the number of buffers that may be used in an I/O operation. Where the number of buffers (with bytes remaining), exceeds this limit, then the I/O operation is performed with the maximum number of buffers allowed by the operating system.

If a timeout is specified and the timeout elapses before the operation completes then it completes with the exception `InterruptedByTimeoutException`. Where a timeout occurs, and the implementation cannot guarantee that bytes have not been read, or will not be read from the channel into the given buffers, then further attempts to read from the channel will cause an unspecific runtime exception to be thrown.

Type Parameters:

`A - The type of the attachment`

Parameters:

`dsts - The buffers into which bytes are to be transferred`

`offset - The offset within the buffer array of the first buffer into which bytes are to be transferred; must be non-negative and no larger than dsts.length`

`length - The maximum number of buffers to be accessed; must be non-negative and no larger than dsts.length - offset`

timeout - The maximum time for the I/O operation to complete

unit - The time unit of the timeout argument

attachment - The object to attach to the I/O operation; can be null

handler - The handler for consuming the result

Throws:

IndexOutOfBoundsException - If the pre-conditions for the offset and length parameter aren't met

IllegalArgumentException - If the buffer is read-only

ReadPendingException - If a read operation is already in progress on this channel

NotYetConnectedException - If this channel is not yet connected

ShutdownChannelGroupException - If the channel group has terminated

write

```
public abstract <A> void write(ByteBuffer src,
                               long timeout,
                               TimeUnit unit,
                               A attachment,
                               CompletionHandler<Integer,? super A> handler)
```

Writes a sequence of bytes to this channel from the given buffer.

This method initiates an asynchronous write operation to write a sequence of bytes to this channel from the given buffer. The `handler` parameter is a completion handler that is invoked when the write operation completes (or fails). The result passed to the completion handler is the number of bytes written.

If a timeout is specified and the timeout elapses before the operation completes then it completes with the exception InterruptedByTimeoutException. Where a timeout occurs, and the implementation cannot guarantee that bytes have not been written, or will not be written to the channel from the given buffer, then further attempts to write
to the channel will cause an unspecific runtime exception to be thrown.

Otherwise this method works in the same manner as the AsynchronousByteChannel.write(ByteBuffer,Object,CompletionHandler) method.

Type Parameters:

A - The type of the attachment

Parameters:

`src` - The buffer from which bytes are to be retrieved

`timeout` - The maximum time for the I/O operation to complete

`unit` - The time unit of the timeout argument

`attachment` - The object to attach to the I/O operation; can be `null`

`handler` - The handler for consuming the result

Throws:

`WritePendingException` - If a write operation is already in progress on this channel

`NotYetConnectedException` - If this channel is not yet connected

`ShutdownChannelGroupException` - If the channel group has terminated

write

```
public final <A> void write(ByteBuffer src,
                            A attachment,
                            CompletionHandler<Integer,? super A> handler)
```

Description copied from interface: `AsynchronousByteChannel`

Writes a sequence of bytes to this channel from the given buffer.

This method initiates an asynchronous write operation to write a sequence of bytes to this channel from the given buffer. The `handler` parameter is a completion handler that is invoked when the write operation completes (or fails). The result passed to the completion handler is the number of bytes written.

The write operation may write up to *r* bytes to the channel, where *r* is the number of bytes remaining in the buffer, that is, `src.remaining()` at the time that the write is attempted. Where *r* is 0, the write operation completes immediately with a result of `0` without initiating an I/O operation.

Suppose that a byte sequence of length *n* is written, where `0` < *n* <= *r*. This byte sequence will be transferred from the buffer starting at index *p*, where *p* is the buffer's position at the moment the write is performed; the index of the last byte written will be *p* + *n* - 1. Upon completion the buffer's position will be equal to *p* + *n*; its limit will not have changed.

Buffers are not safe for use by multiple concurrent threads so care should be taken to not access the buffer until the operation has completed.

This method may be invoked at any time. Some channel types may not allow more than one write to be outstanding at any given time. If a thread initiates

a write operation before a previous write operation has completed then a WritePendingException will be thrown.

Specified by:

write in interface AsynchronousByteChannel

Type Parameters:

A - The type of the attachment

Parameters:

src - The buffer from which bytes are to be retrieved

attachment - The object to attach to the I/O operation; can be null

handler - The completion handler object

Throws:

WritePendingException - If the channel does not allow more than one write to be outstanding and a previous write has not completed

NotYetConnectedException - If this channel is not yet connected

ShutdownChannelGroupException - If the channel group has terminated

write

```
public abstract Future<Integer> write(ByteBuffer src)
```

Description copied from interface: AsynchronousByteChannel

Writes a sequence of bytes to this channel from the given buffer.

This method initiates an asynchronous write operation to write a sequence of bytes to this channel from the given buffer. The method behaves in exactly the same manner as the write(ByteBuffer,Object,CompletionHandler) method except that instead of specifying a completion handler, this method returns a Future representing the pending result. The Future's get method returns the number of bytes written.

Specified by:

write in interface AsynchronousByteChannel

Parameters:

src - The buffer from which bytes are to be retrieved

Returns:

A Future representing the result of the operation

Throws:

WritePendingException - If the channel does not allow more than one write to be outstanding and a previous write has not completed

NotYetConnectedException - If this channel is not yet connected

write

```
public abstract <A> void write(ByteBuffer[] srcs,
                               int offset,
                               int length,
                               long timeout,
                               TimeUnit unit,
                               A attachment,
                               CompletionHandler<Long,? super A> handler)
```

Writes a sequence of bytes to this channel from a subsequence of the given buffers. This operation, sometimes called a *gathering write*, is often useful when implementing network protocols that group data into segments consisting of one or more fixed-length headers followed by a variable-length body. The `handler` parameter is a completion handler that is invoked when the write operation completes (or fails). The result passed to the completion handler is the number of bytes written.

This method initiates a write of up to *r* bytes to this channel, where *r* is the total number of bytes remaining in the specified subsequence of the given buffer array, that is,

```
srcs[offset].remaining()
    + srcs[offset+1].remaining()
    + ... + srcs[offset+length-1].remaining()
```

at the moment that the write is attempted.

Suppose that a byte sequence of length *n* is written, where `0` < *n* <= *r*. Up to the first `srcs[offset].remaining()` bytes of this sequence are written from buffer `srcs[offset]`, up to the next `srcs[offset+1].remaining()` bytes are written from buffer `srcs[offset+1]`, and so forth, until the entire byte sequence is written. As many bytes as possible are written from each buffer, hence the final position of each updated buffer, except the last updated buffer, is guaranteed to be equal to that buffer's limit. The underlying operating system may impose a limit on the number of buffers that may be used in an I/O operation. Where the number of buffers (with bytes remaining), exceeds this limit, then the I/O operation is performed with the maximum number of buffers allowed by the operating system.

If a timeout is specified and the timeout elapses before the operation completes then it completes with the exception `InterruptedByTimeoutException`. Where a timeout occurs, and the implementation cannot guarantee that bytes have not been written, or will not be written to the channel from the given buffers, then further attempts to write to the channel will cause an unspecific runtime exception to be thrown.

Type Parameters:

A - The type of the attachment

Parameters:

srcs - The buffers from which bytes are to be retrieved

offset - The offset within the buffer array of the first buffer from which bytes are to be retrieved; must be non-negative and no larger than srcs.length

length - The maximum number of buffers to be accessed; must be non-negative and no larger than srcs.length - offset

timeout - The maximum time for the I/O operation to complete

unit - The time unit of the timeout argument

attachment - The object to attach to the I/O operation; can be null

handler - The handler for consuming the result

Throws:

IndexOutOfBoundsException - If the pre-conditions for the offset and length parameter aren't met

WritePendingException - If a write operation is already in progress on this channel

NotYetConnectedException - If this channel is not yet connected

ShutdownChannelGroupException - If the channel group has terminated

getLocalAddress

```
public abstract SocketAddress getLocalAddress()
                                        throws IOException
```

Returns the socket address that this channel's socket is bound to.

Where the channel is bound to an Internet Protocol socket address then the return value from this method is of type InetSocketAddress.

If there is a security manager set, its checkConnect method is called with the local address and -1 as its arguments to see if the operation is allowed. If the operation is not allowed, a SocketAddress representing the loopback address and the local port of the channel's socket is returned.

Specified by:

getLocalAddress in interface NetworkChannel

Returns:

The SocketAddress that the socket is bound to, or the SocketAddress representing the loopback address if denied by the security manager, or null if the channel's socket is not bound

Throws:

`ClosedChannelException` - If the channel is closed

`IOException` - If an I/O error occurs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.